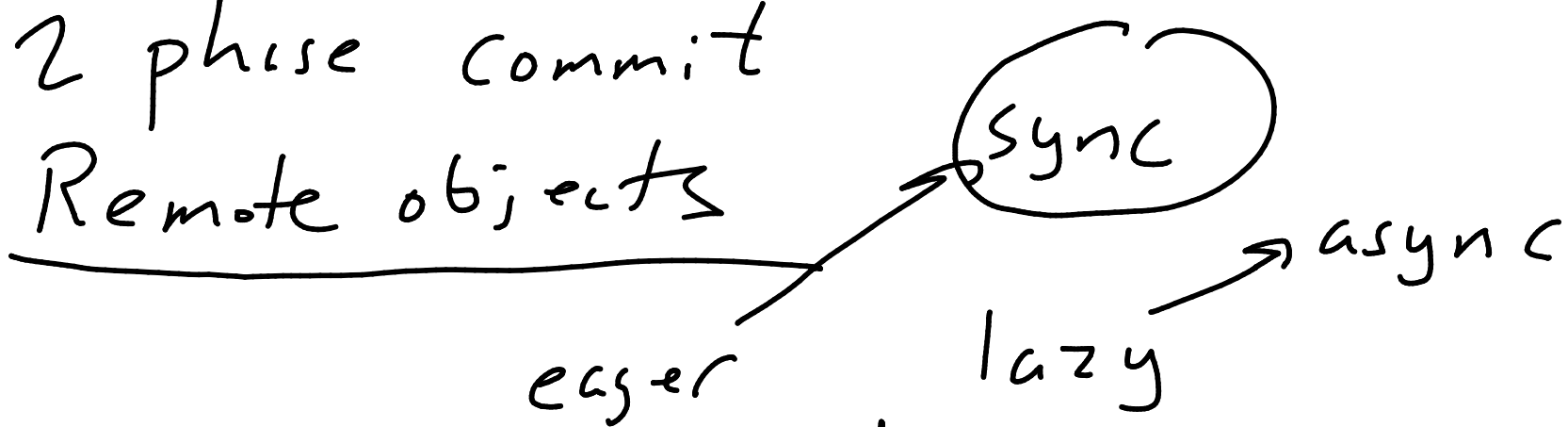


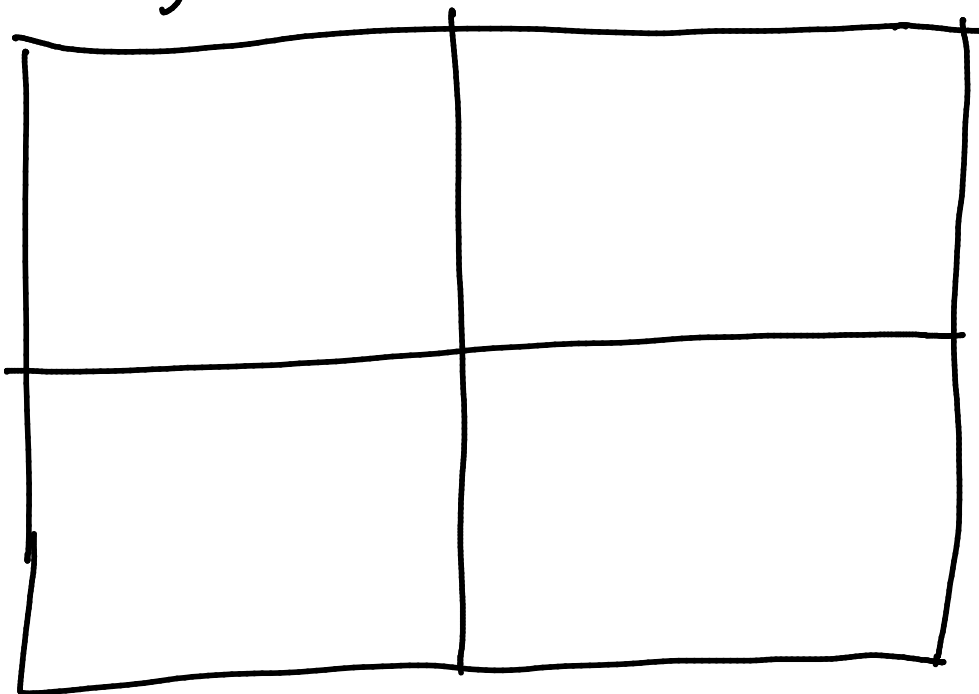
2 phase commit

Remote objects



writes
only
on primary

writes
everywhere



write



way to make

sure that update

happens everywhere, "consistently"

- everyone perceives the change

as happening everywhere

"at once"

Computers

I can't lock another computer

Last time: 2 general's problem

- no guaranteed protocol in finite time to guarantee two computers could agree to make an update

2 phase commit protocol

Assumptions

- no node crashes forever
- each node has a "write ahead log" (WAL) that is stable, never corrupted, and never lost

Phase 1: node w/ the update to make (primary) sends a "prepare to commit" message to all other nodes, including itself, along with the update

All the nodes execute the transaction locally without committing (no one else sees those updates)
- then nodes write what they've done to the WAL, then they block responses to anyone, and respond to primary "ready to commit"

Phase 2:

If the primary hears "ready to commit" from everyone (including self), then it tells everyone "commit"

All the nodes commit, then unblock.

All the nodes communicate "ack" to primary

When all have done so, primary unblocks and says done

If at any time, a replica aborts
(in phase 1), the primary
tells all to abort.

Suppose I told you to go code
this. What technology are you using?

MPI x (?)

↓ Spark x

Suppose I wanted to write more
general purpose code

- sockets - send bytes to other
computers

Middle level: remote procedure calls
remote method invocation
remote objects

All of kinds of challenges prevent themselves

- There are good solns, there are popular libraries to do this